Seção 2: Arquitetura e Conexões

Esta seção descreve a arquitetura de software proposta para o aplicativo Finan, detalhando os componentes do frontend, backend e a forma como eles se conectam e interagem. A arquitetura visa ser escalável, segura e de fácil manutenção, utilizando tecnologias modernas e adequadas aos requisitos do projeto.

Arquitetura Frontend

O frontend do Finan será desenvolvido como um aplicativo móvel multiplataforma, priorizando uma experiência de usuário fluida e responsiva em dispositivos iOS e Android.

- **Frameworks e Bibliotecas Principais:** A base do desenvolvimento frontend será o **React Native** em conjunto com o **Expo**. Essa combinação foi escolhida por permitir o desenvolvimento rápido, o acesso a uma vasta gama de APIs nativas através do Expo SDK e a capacidade de gerar builds para ambas as plataformas a partir de um único código-base. Para componentes visuais e de interface, além dos componentes nativos do React Native, poderemos considerar o uso de bibliotecas como **React Native Elements** ou **NativeBase** para acelerar o desenvolvimento de interfaces estilizadas e consistentes, caso seja avaliado como benéfico para a produtividade e qualidade visual. Para gráficos, a biblioteca `react-native-chart-kit` já foi mencionada no README e será utilizada inicialmente, com possibilidade de explorar outras como `VictoryNative` para visualizações mais complexas ou personalizadas nas fases posteriores.

- **Estrutura de Pastas Sugerida:** Uma estrutura de pastas organizada é crucial para a manutenibilidade do projeto. Baseando-se nas convenções comuns em projetos React Native e na estrutura inicial observada no repositório, propõe-se a seguinte organização principal dentro da pasta `src` (ou `app` como no repositório original):

    - `screens/`: Contendo os componentes de tela, cada um em sua respectiva subpasta se a complexidade aumentar (ex: `screens/HomeScreen/`, `screens/TransactionsScreen/`).
    - `components/`: Para componentes reutilizáveis da UI (botões, inputs, cards, etc.). Poderão ser subdivididos em `common/` para componentes genéricos e `features/` para componentes específicos de alguma funcionalidade.
    - `navigation/`: Configuração dos navegadores (stack, tab, drawer) utilizando **React Navigation**.

    - `services/` : Módulos responsáveis pela comunicação com o backend (Firebase) e outras APIs externas.
    - `store/` ou `context/` : Para gerenciamento de estado global. Dependendo da complexidade, pode-se iniciar com a **Context API** do React para estados mais simples e localizados, e considerar a adoção de **Redux (com Redux Toolkit)** ou **Zustand** se a complexidade do estado global crescer significativamente, especialmente com funcionalidades como sincronização de dados em tempo real e caching.
    - `hooks/` : Hooks customizados para lógica reutilizável.
    - `utils/` : Funções utilitárias diversas (formatação de datas, validações, etc.).
    - `assets/` : Imagens, fontes e outros arquivos estáticos.
    - `constants/` : Constantes utilizadas em todo o aplicativo (chaves de API, strings de configuração, etc.).
    - `config/` : Arquivos de configuração, como a inicialização do Firebase.

- **Gerenciamento de Estado:** Inicialmente, para o MVP, a **React Context API** combinada com `useState` e `useReducer` pode ser suficiente para gerenciar o estado global, como informações do usuário autenticado e dados de transações recentes. À medida que o aplicativo crescer, especialmente com a introdução de funcionalidades como sincronização em tempo real, planejamento de contas e relatórios complexos, a migração para uma biblioteca de gerenciamento de estado mais robusta como **Redux Toolkit** (pela sua simplicidade e eficiência) ou **Zustand** (pela sua leveza e API minimalista) será avaliada. A escolha visará manter a previsibilidade do estado e facilitar o debugging.

- **Componentes Principais:** Alguns componentes principais, além das telas, incluirão:

    - `TransactionForm` : Modal ou bottom sheet para entrada de dados de despesas/receitas.
    - `TransactionListItem` : Componente para exibir um item individual na lista de transações.
    - `CategoryPicker` : Seletor de categorias.
    - `DatePicker` : Seletor de datas.
    - `BudgetCard` : Componente para exibir o status de um orçamento.
    - `GoalProgress` : Componente para visualizar o progresso de uma meta de economia.
    - `ChartDisplay` : Componente genérico para encapsular a lógica de exibição dos diferentes tipos de gráficos.

Arquitetura Backend

O backend do Finan será construído utilizando a plataforma **Firebase** como um Backend-as-a-Service (BaaS). Esta escolha simplifica o desenvolvimento e a manutenção da infraestrutura de backend, permitindo que a equipe foque na lógica de negócios do aplicativo.

- **Linguagens e Frameworks:** O Firebase em si não exige a escolha de uma linguagem de backend tradicional para muitas de suas funcionalidades. No entanto, para lógica de backend customizada que não pode ser implementada diretamente no cliente ou através das regras de segurança do Firestore, serão utilizadas **Cloud Functions for Firebase**. Estas funções são tipicamente escritas em **Node.js** (com JavaScript ou TypeScript). TypeScript é recomendado para maior robustez e manutenibilidade do código das Cloud Functions.

- **Padrões de API:** O Firebase SDK para cliente (React Native) abstrai a comunicação direta via REST ou GraphQL para a maioria das operações com o Firestore e Authentication. As interações ocorrem através de chamadas de método do SDK. Para as Cloud Functions que precisarem ser expostas como endpoints HTTP (por exemplo, para integrações futuras ou webhooks), elas podem ser configuradas para seguir princípios **RESTful**.

- **Autenticação:** Será utilizado o **Firebase Authentication**. Ele oferece suporte para autenticação via e-mail/senha, provedores sociais (Google, Facebook, etc., que podem ser considerados para fases futuras) e gerenciamento de sessão de usuário. As regras de segurança do Firebase serão configuradas para garantir que apenas usuários autenticados possam acessar e modificar seus próprios dados.

- **Banco de Dados:** O **Cloud Firestore** será o banco de dados principal. É um banco de dados NoSQL, orientado a documentos, que oferece sincronização em tempo real, consultas poderosas e escalabilidade automática. A estrutura de dados no Firestore será modelada para otimizar as consultas comuns do aplicativo, como listagem de transações por período, filtragem por categoria e agregação de dados para relatórios. Por exemplo:

    - Coleção `users`: Armazena informações do perfil do usuário.
    - Coleção `transactions`: Subcoleção de cada usuário, armazenando todas as suas transações financeiras, com campos como `userId`, `type` (despesa/receita), `amount`, `category`, `date`, `description`, `receiptImageUrl` (opcional).
    - Coleção `budgets`: Subcoleção de cada usuário, para os orçamentos definidos.

    - Coleção `goals`: Subcoleção de cada usuário, para as metas de economia.
    - Coleção `categories`: Pode ser uma coleção global com categorias padrão e uma subcoleção por usuário para categorias personalizadas.

Integração e Fluxo de Dados

A integração entre o frontend (React Native) e o backend (Firebase) será realizada predominantemente através do Firebase SDK.

- **Fluxo de Dados:**
    1. **Autenticação:** O usuário se registra ou faz login no aplicativo. O Firebase Authentication SDK envia as credenciais para o Firebase, que valida e retorna um token de sessão. O estado de autenticação é gerenciado no frontend.
    2. **Operações CRUD (Create, Read, Update, Delete):** Para funcionalidades como registrar uma transação, o frontend utiliza o Firestore SDK para escrever o novo documento na coleção apropriada. As leituras de dados (ex: buscar transações) são feitas com consultas ao Firestore, que podem ser configuradas para ouvir atualizações em tempo real, refletindo as mudanças instantaneamente na UI.
    3. **Cloud Functions:** Para operações que exigem lógica de servidor mais complexa ou acesso privilegiado (ex: processamento de notificações bancárias, geração de relatórios agregados complexos, envio de notificações push agendadas), o frontend pode invocar uma Cloud Function via HTTPS ou através de gatilhos do Firestore (ex: uma nova transação é criada e uma função é acionada para atualizar um resumo).
- **Endpoints Necessários (Conceitual):** Embora o Firebase SDK abstraia os endpoints HTTP, conceitualmente, as interações representam:
    - `POST /users/{userId}/transactions`: Criar nova transação.
    - `GET /users/{userId}/transactions?dateRange=...&category=...`: Ler transações com filtros.
    - `PUT /users/{userId}/transactions/{transactionId}`: Atualizar transação.
    - `DELETE /users/{userId}/transactions/{transactionId}`: Deletar transação.
    - Chamadas a Cloud Functions específicas para funcionalidades como `processBankNotification` ou `generateMonthlyReport`.
- **Protocolos:** Toda a comunicação entre o cliente e o Firebase ocorre sobre **HTTPS**, garantindo a criptografia dos dados em trânsito.

- **Tratamento de Erros:** O tratamento de erros será implementado em ambas as camadas:
    - **Frontend:** Utilizar blocos `try-catch` para chamadas assíncronas ao Firebase. Exibir mensagens de erro amigáveis para o usuário e registrar erros mais detalhados para debugging (possivelmente usando um serviço como Sentry ou Firebase Crashlytics). Implementar retentativas para falhas de rede temporárias.
    - **Backend (Cloud Functions):** Validar dados de entrada. Utilizar tratamento de erros robusto dentro das funções e retornar códigos de status HTTP apropriados e mensagens de erro claras em caso de falha. Logs detalhados serão enviados para o Google Cloud Logging.
    - **Regras de Segurança do Firestore:** Serão extensivamente utilizadas para prevenir operações não autorizadas ou malformadas no nível do banco de dados, atuando como uma primeira linha de defesa.

Esta arquitetura fornece uma base sólida para o desenvolvimento do Finan, equilibrando a velocidade de desenvolvimento com a escalabilidade e segurança necessárias para um aplicativo financeiro.